Floating on Main: Detecting Unpinned Model Dependencies in MLOps

Aditya Vardhan Khetawat
Dept. of Artificial Intelligence & Machine Learning
Manipal University Jaipur
Jaipur, Rajasthan, India
aditya.23fe10cai00586@muj.manipal.edu

Rishi Sharma
Dept. of Artificial Intelligence & Machine Learning
Manipal University Jaipur
Jaipur, Rajasthan, India
rishi.23fe10cai00587@muj.manipal.edu

Samairaa Arora
Dept. of Artificial Intelligence & Machine Learning
Manipal University Jaipur
Jaipur, Rajasthan, India
samairaa.23fe10cai00439@muj.manipal.edu

Dr. Manish Rai
Dept. of Artificial Intelligence & Machine Learning
Manipal University Jaipur
Jaipur, Rajasthan, India
manish.rai@jaipur.manipal.edu

***Abstract*—Pre-trained models (PTMs) from hubs such as Hugging Face are loaded by name at run time; unless a revision is pinned, each new environment receives whatever the owner last pushed, undermining the reproducibility that MLOps requires. We present PinCheck, a static analyzer that detects model-loading call sites across seven Hugging Face interfaces, resolves identifiers through intra-file constant propagation, and classifies revision specifications. On 2,026 candidate call sites from 60 held-out repositories, PinCheck reaches 99.8% detection precision versus 90.2% for a line-based pattern-matching baseline, which recovers only 8.5% of pinned revisions and 18.6% of hub identifiers and would underestimate the pinning rate by a factor of thirteen. Applying PinCheck to 939 actively maintained GitHub repositories (12,898 call sites in 589 projects), we find that 84.9% of projects never specify a model revision and only 1.7% pin an immutable commit. Of the projects that lock their library dependencies, 82.1% leave every model floating. One in five projects executes repository code via `trust_remote_code` without pinning it. Pinning increases with MLOps maturity (Cochran–Armitage $z = 4.24$, $p < 0.001$), and automated testing and popularity remain associated with it after adjustment. We release PinCheck and propose a CI-enforced model lockfile.**

***Index Terms*—MLOps, pre-trained models, dependency management, static analysis, reproducibility, software supply chain**

I. Introduction

MLOps aims to make machine learning systems versioned, testable, and reproducible across their lifecycle [1], [2]. For code dependencies, Python projects achieve this through version constraints and lockfiles. Pre-trained models, however, have become dependencies of a different kind [3]. A call such as `from_pretrained("org/model")` resolves the default branch of a Git repository on Hugging Face *at download time*. Model owners routinely push weight, tokenizer, template, and configuration changes in place [4], [5], and repositories can disappear: `runwayml/stable-diffusion-v1-5` was removed, breaking downstream code [6]. Unless the loader passes a `revision` (ideally a commit hash), a project with fixed code and fixed library versions can still change behavior silently (Fig. 1).

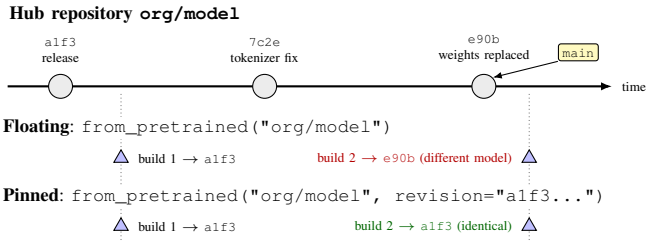


Fig. 1. A floating reference resolves the default branch (`main`) at download time, so two builds of unchanged code can load different models. A pinned commit hash always resolves to the same snapshot.

A recent manual study of 401 projects found that only 12% specify a PTM version [3]. Manual analysis does not scale, however, and our experiments show that the pattern-matching techniques common in repository mining misjudge this property badly. This paper makes four contributions:

1) **PinCheck**, an AST-based detector for PTM loads that resolves identifiers via constant propagation and classifies revisions, evaluated on a held-out sample against a pattern-matching baseline, with an ablation of constant propagation.
2) A measurement of model pinning in 939 actively maintained repositories (12,898 call sites), including per-interface rates and *pinning coverage* within projects.
3) Evidence of a *pinning asymmetry* between library and model dependencies, and of widespread unpinned remote code execution.
4) A multivariable analysis linking model pinning to MLOps maturity signals, and a CI-enforceable model lockfile workflow.

TABLE I
COMPARISON WITH CLOSEST PRIOR WORK

	[3]	[4], [5]	**This work**
Perspective	Consumer	Producer	Consumer
Method	Manual	Mining	Evaluated static
Scale	401 projects	Hub models	939 repos, 12.9k sites
Pinning measured	Project	–	Call site + project
Code vs. model pinning	No	No	Yes
Remote-code exposure	No	No	Yes
Link to MLOps practices	No	No	Yes

II. RELATED WORK

MLOps and technical debt. Sculley et al. characterized ML-specific technical debt, including unstable data dependencies [2]; the ML Test Score lists reproducibility as a production-readiness criterion [7]; and Kreuzberger et al. define versioning of code, data, and models as a core MLOps principle [1]. Calefato et al. mined MLOps practices such as CI adoption on GitHub [8], but did not consider PTM dependencies.

PTM reuse and evolution. Jiang et al. studied reuse of Hugging Face models [9], and PeaTMOSS linked GitHub projects to the PTMs they use [10]. Yasmin et al. framed PTMs as "Software Dependencies 2.0" and reported, by manual analysis, that 12% of 401 projects specify a model version [3]. On the producer side, model weights change without explicit version signals [4], and model repositories keep evolving after release [5].

Consequences of model change. Updates introduce "negative flips" even when accuracy improves [11]; the behavior of LLM services drifts between versions [12], motivating regression testing for evolving models [13]. Model hubs also host malicious code [14], which matters because `trust_remote_code` executes code fetched from the model repository. *Dependency management and reproducibility.* Version-constraint practices [15] and update lag [16] are well studied in package ecosystems, where lockfiles record exact versions and releases are immutable; a model's default branch, by contrast, can be rewritten at any time. Unspecified dependencies also commonly break notebook re-execution [17].

Positioning. Table I contrasts this work with the closest studies. We differ in providing an evaluated automated detector, call-site granularity, current data, and analyses of code–model asymmetry, remote code, and maturity that prior work does not report.

III. PINCHECK: DETECTING MODEL DEPENDENCIES

PinCheck parses each Python file and notebook (code cells, with shell and magic lines removed) into an abstract syntax tree and visits every call. A call is a *model-loading site* if its callee is `from_pretrained` (Transformers, Diffusers, PEFT), `SentenceTransformer`, `CrossEncoder`, `hf_hub_download`, `snapshot_download`, LangChain's `HuggingFaceEmbeddings`, or `pipeline` when imported from Transformers. Method calls on objects that happen to be named `pipeline` are excluded.

Algorithm 1 PinCheck call-site classification

Require: source file S
$T \leftarrow$ PARSE(S); $K \leftarrow$ names bound to exactly one string constant in T
$I \leftarrow$ **true** if T imports `pipeline` from `transformers`
for each call $n \in T$ whose callee $\in$ the seven interfaces **do**
  if callee is `pipeline` **and** ($\neg I$ **or** n is a method call) **then**
    continue
  end if
  $a \leftarrow$ identifier argument (positional or keyword); $v \leftarrow$ RESOLVE(a, K)
  if a absent **and** callee is `pipeline` **then**
    $id \leftarrow$ DEFAULT
  else if v undefined **then**
    $id \leftarrow$ INDIRECT
  else if v is a path, file name, or path prefix **then**
    $id \leftarrow$ LOCAL ▷ excluded
  else
    $id \leftarrow$ HUB
  end if
  $\rho \leftarrow$ `revision` keyword (or in `model_kwargs`); $w \leftarrow$ RESOLVE(ρ, K)
  if ρ absent **then**
    $rev \leftarrow$ NONE
  else if w matches `[0-9a-f]{40}` **then**
    $rev \leftarrow$ HASH
  else if w is a literal **then**
    $rev \leftarrow$ NAMEDREF
  else
    $rev \leftarrow$ CONFIGURABLE
  end if
  emit ($line, callee, id, rev,$ `trust_remote_code`)
end for

Identifier resolution. The identifier argument is located by position or keyword. If it is a string literal, or a name assigned exactly one string constant in the file (*intra-file constant propagation*), it is classified as a *hub literal* unless it is a path, a file name, or a path-prefixed concatenation, which are *local* and excluded. Unresolvable expressions are *indirect*; `pipeline(task)` without a model is a *default pipeline*.

Revision classification. The `revision` keyword (also inside LangChain's `model_kwargs`) is resolved in the same way and classified as a 40-hex *hash* (immutable), a *named reference* such as `"main"` (mutable), *configurable* (a variable), or *none*. PinCheck also records whether `trust_remote_code=True` is set.

Algorithm 1 summarizes the procedure. In the algorithm, T is the syntax tree of the file, K maps variable names to their unique string values, I indicates a Transformers `pipeline` import, a and v are the identifier argument and its resolved value, and ρ and w are the `revision` argument and its resolved value.

Design rationale. Three choices distinguish PinCheck from text search. Parsing to an AST separates executable calls from docstrings, comments, and definitions. Resolving keyword arguments on the AST finds `revision` wherever it appears in a multi-line call. Intra-file constant propagation recovers identifiers that projects keep in module-level constants, a common style for model configuration. Restricting `pipeline` to the Transformers import avoids collisions with project functions

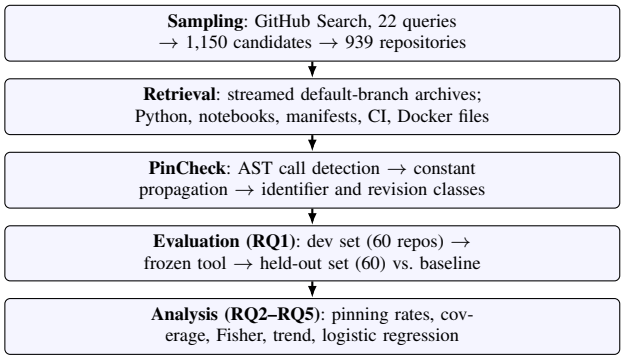


Fig. 2. Study pipeline, from repository sampling to the held-out detector evaluation and statistical analysis.

and pipeline objects. These choices keep the analysis linear in file size, and the full corpus was processed on a single CPU core with 3 GB of memory.

Manifest analysis. For `transformers`, `torch`, `sentence-transformers`, and `huggingface-hub`, PinCheck records the strongest declaration in any manifest (`requirements*.txt`, `pyproject.toml`, `setup.py`, `setup.cfg`, `Pipfile`, `environment.yml`) as exact, range, unpinned, or undeclared, and detects lockfiles (Poetry, uv, Pipenv, PDM), GitHub Actions workflows, test files, and Dockerfiles.

IV. Study Design

Research questions. **RQ1:** How accurately does PinCheck detect model loads and revisions compared with pattern matching? **RQ2:** How prevalent is model pinning? **RQ3:** Do projects that pin libraries also pin models? **RQ4:** How common are high-risk loading patterns? **RQ5:** Is pinning associated with MLOps maturity?

Sampling. Fig. 2 outlines the pipeline. We queried the GitHub Search API with ten PTM-related topics (e.g., `huggingface`, `transformers`, `rag`, `semantic-search`) and twelve application keyword queries, keeping Python repositories with $\geq$20 stars, a push since 1 January 2025, and no fork or archive flag. Removing repositories over 80 MB and the producers of the loading libraries left 939 repositories. Default-branch archives were analyzed on 20 October 2026 (one repository had been deleted); vendored library copies and virtual environments were excluded. In total, 138,940 Python files and 970 notebooks were parsed.

Metrics. Let C be a set of model-loading call sites and, for a site $c \in C$, let $r(c)$ be its `revision` argument, with $r(c) = \varnothing$ when no revision is given. The *pinning rate* of C is

$$\pi(C) = \frac{|\{c \in C : r(c) \neq \varnothing\}|}{|C|}, \tag{1}$$

where $|\cdot|$ denotes the number of elements of a set. For a repository P with call sites C_P, we define the pinning indicator and the pinning coverage

$$\mathrm{pin}(P) = \begin{cases} 1 & \text{if } \pi(C_P) > 0 \\ 0 & \text{otherwise,} \end{cases} \qquad \kappa(P) = \pi(C_P). \tag{2}$$

Thus $\mathrm{pin}(P) = 1$ if at least one load in P specifies a revision, which gives an upper bound on pinning, and $\kappa(P)$ is the fraction of P's loads that do. The *strict* indicator $\mathrm{pin}^*(P)$ counts only sites whose $r(c)$ is a literal hash or named reference. A repository is *library-pinned* if any of the four key packages is pinned with == or a lockfile exists. The MLOps maturity score is

$$M(P) = \mathbf{1}_{\mathrm{CI}}(P) + \mathbf{1}_{\mathrm{Test}}(P) + \mathbf{1}_{\mathrm{Docker}}(P) + \mathbf{1}_{\mathrm{LibPin}}(P), \tag{3}$$

where each indicator $\mathbf{1}_X(P)$ equals 1 if P has property X and 0 otherwise: a GitHub Actions workflow (CI), test files (Test), a Dockerfile or Compose file (Docker), and library pinning (LibPin). Hence $M(P) \in \{0, 1, 2, 3, 4\}$.

Statistics. For a binary signal X, the odds ratio is

$$\mathrm{OR} = \frac{a/b}{c/d}, \tag{4}$$

where a and b are the numbers of pinning and non-pinning repositories with X, and c and d those without X. $\mathrm{OR} > 1$ means pinning is more likely when X is present. Significance is assessed with two-sided Fisher exact tests, using a Bonferroni-corrected threshold of $\alpha = 0.05/4 = 0.0125$ for the four signals. A Cochran–Armitage test checks for a linear trend in the pinning proportion across the ordered levels of M, and Mann–Whitney U tests compare star counts and ages. To separate correlated signals, we fit the logistic regression

$$\ln \frac{p_P}{1 - p_P} = \beta_0 + \sum_{k=1}^{6} \beta_k x_k(P), \tag{5}$$

where $p_P = \Pr(\mathrm{pin}(P) = 1)$, $x_1, \ldots, x_4$ are the CI, Test, Docker, and LibPin indicators, $x_5 = \log_{10}$ of the star count, and x_6 is the repository age in years. The adjusted odds ratio e^{β_k} is the factor by which the odds of pinning change per unit increase in x_k with the other predictors held fixed. Model fit is summarized by McFadden's

$$R^2_{\mathrm{McF}} = 1 - \frac{\ln \hat{L}_{\mathrm{full}}}{\ln \hat{L}_0}, \tag{6}$$

where $\hat{L}_{\mathrm{full}}$ and $\hat{L}_0$ are the maximized likelihoods of the fitted and the intercept-only models.

Detector evaluation (RQ1). PinCheck was developed on a sample of 60 repositories (459 files). The detector was then frozen and run, together with a baseline, on a disjoint held-out sample of 60 repositories (819 files). The *baseline* reflects the line-based pattern matching common in repository mining: a regular expression matches the seven callees on non-comment lines, takes the first quoted argument as the identifier, and marks a site as pinned if `revision` appears on the same line. Ground truth was built by adjudicating every disagreement between the two tools. For each task we report

$$\mathrm{Precision} = \frac{TP}{TP + FP}, \qquad \mathrm{Recall} = \frac{TP}{TP + FN}, \tag{7}$$

TABLE II
HELD-OUT DETECTOR EVALUATION (60 REPOSITORIES)

Measure	Baseline	PinCheck w/o CP	PinCheck
Detection precision	90.2%	99.8%	99.8%
Detection recall†	100%	100%	100%
Hub-literal recall (436)	18.6%	30.3%	100%
Hub-literal precision	45.3%	100%	100%
Revision recall (94)	8.5%	100%	100%
Estimated pinning rate	0.4%	5.2%	5.2%

†Relative to the union of both tools' detections; sites missed by both cannot be counted. CP = constant propagation.

TABLE III
CAUSES OF BASELINE ERRORS ON THE HELD-OUT SAMPLE

Cause	Sites	Effect on baseline
Identifier via constant	304	Hub literal missed
`revision` on later line	86	Pinning missed
Identifier on later line	51	Hub literal missed
Match inside docstring / string	117	False site
Function definition	74	False site
Object invocation, message, class	4	False site
Project-defined `pipeline`	3	False site (both tools)

where TP counts correctly reported items (call sites, hub literals, or pinned revisions), FP counts reported items that are wrong, and FN counts true items that were not reported. Disagreements were grouped by cause, and at least eight randomly drawn cases per cause (all cases for causes with fewer than eight) were inspected manually (definition matches were identified syntactically). In addition, 30 randomly drawn agreements were inspected; all were correct.

V. RESULTS

A. RQ1: Detector Accuracy

Table II summarizes the held-out evaluation over 2,026 candidate sites. The baseline produced 198 false detections: 117 matches inside docstrings and documentation strings (all non-executable in the held-out set), 74 function definitions such as `def from_pretrained(`, 3 other non-call matches (messages and a class definition), 1 invocation of a pipeline object, and 3 calls to a project-defined `pipeline` function. PinCheck's only errors were the last 3, caused by a name collision with the Transformers `pipeline`. We corrected this after the evaluation by requiring an import from Transformers, and the corrected version was used for RQ2–RQ5.

The baseline's deeper failure is classification. It finds only 8 of the 94 pinned revisions, because 86 appear on a later line of a multi-line call, and only 81 of the 436 hub literals, because 304 are reached through constants (`MODEL_ID = "..."`) and 51 sit on a following line. Its own hub-literal matches are mostly documentation examples (98 of 179). Consequently, the baseline would estimate the call-level pinning rate π in (1) at 0.4% instead of 5.2%, an underestimation factor of $5.2/0.4 \approx 13$. Without constant propagation, PinCheck recovers only 30.3% of hub literals, showing that constant resolution is essential for identifying which model a project loads.

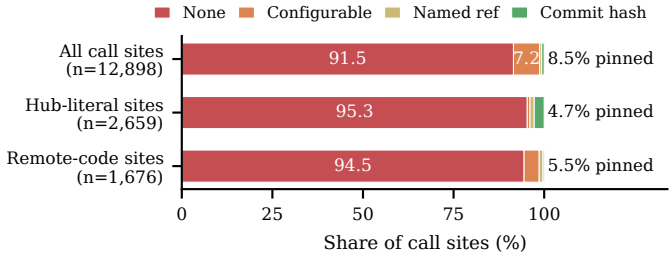


Fig. 3. Revision status of call sites (numbers inside bars are percentages). Fewer than 6% of hub-literal and remote-code sites specify any revision, and fewer than 3% pin a commit hash.

Answer to RQ1: Syntax-aware analysis is necessary. Pattern matching overcounts sites, misses most identifiers and revisions, and would distort prevalence estimates by an order of magnitude.

B. RQ2: Prevalence of Model Pinning

Of the 939 repositories, 589 contain at least one non-local model load (median 245 stars, range 20–75,310). They hold 12,898 call sites referencing 825 distinct hub identifiers. Of these call sites, 78.6% use indirect identifiers, 20.6% hub literals, and 0.7% no identifier. Of all call sites, 91.5% specify no revision, 7.2% a configurable one, 0.6% a named reference, and 0.7% a hash. At the project level, 500 repositories (84.9%) never specify a revision, 63 (10.7%) have only configurable revisions, 16 (2.7%) use a named reference, and only 10 (1.7%) pin any load to a hash. Just 6 (1.0%) pin every load. Of the 2,659 hub-literal sites, 95.3% are unpinned, and only 19 of 308 repositories with hub literals (6.2%) pin any of them. Fig. 3 compares the revision status of all sites, hub-literal sites, and remote-code sites.

Pinning depends strongly on the interface (Fig. 4a): 32.4% of repositories using `snapshot_download` and 25.2% using `hf_hub_download` pin at least one such call, against 12.0% for `from_pretrained`, 4.7% for `SentenceTransformer`, 1.4% for `pipeline`, and none of 26 repositories for `HuggingFaceEmbeddings`. Pinning is also partial: the median coverage among pinning repositories is 16.4%, so a typical pinning project protects about one in six of its loads. The most common unpinned identifiers are `bert-base-uncased` (43 repositories), `all-MiniLM-L6-v2` (29), and `gpt2` (20); seven repositories still load the removed `runwayml/stable-diffusion-v1-5`.

Identifier characteristics. Of the 308 repositories with hub literals, 52.9% load more than one model. In 281 repositories (47.7%) every identifier is indirect, so the loaded model cannot be determined from code. Pinning is lower in notebooks (4.4%) than in modules (8.6%).

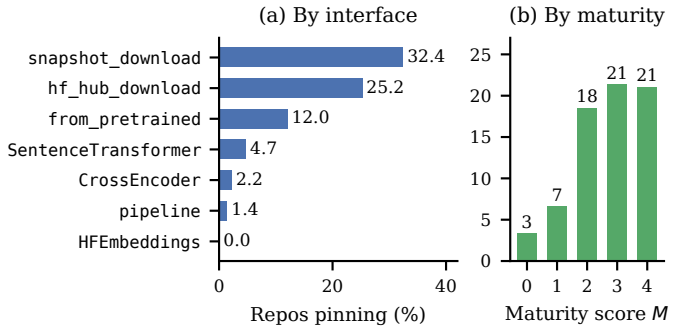


Fig. 4. Share of repositories specifying any revision: (a) on each loading interface; (b) by MLOps maturity score M (n = 61, 137, 184, 131, 76).

C. RQ3: Pinning Asymmetry

`transformers` is pinned exactly in 21.9% of repositories and declared with a range in 31.1%; 21.7% have a lockfile, and 46.5% (274) are library-pinned overall. Of these 274 repositories, 225 (82.1%) pin no model; the figure is 76.7% for projects that pin `transformers` exactly and 85.2% for projects with a lockfile. Library pinning is not significantly associated with model pinning (17.9% vs. 12.7%, OR $= 1.50$, $p = 0.085$). The pattern holds for every key package: `torch` is pinned exactly in 20.4% of repositories, `huggingface-hub` in 10.5%, and `sentence-transformers` in 7.8%, compared with 15.1% of repositories that specify any model revision and 1.7% that pin a hash.

D. RQ4: High-Risk Patterns

In total, 128 repositories (21.7%) enable `trust_remote_code=True` across 1,676 call sites. Of those sites, 94.5% specify no revision and only 6 pin a hash (in 4 repositories). In 85.6% of them the identifier is indirect, so the repository whose code is executed is chosen at run time. Almost all remote-code loads use `from_pretrained` (1,613 sites), mostly for recent models with custom architectures. Default pipelines, whose model is chosen by the installed library version, are rare (7 repositories, 1.2%).

E. RQ5: Association With MLOps Maturity

Pinning is more common with CI (19.4% vs. 10.0%, OR $= 2.17$, $p = 0.002$) and automated tests (18.2% vs. 7.9%, OR $= 2.61$, $p = 0.001$), both significant after correction. The association with Docker is weaker (OR $= 1.67$, $p = 0.029$) and does not survive correction. Pinning rises from 3.3% at $M = 0$ to 21.4% at $M = 3$ (Fig. 4b; $z = 4.24$, $p < 0.001$), and pinning repositories are more popular (median 597 vs. 218 stars, $p < 0.001$). Table IV shows that, after adjustment, only automated tests and popularity remain significant. CI loses its effect because it is correlated with tests ($r = 0.46$). Under the strict definition (26 repositories), Docker (OR $= 3.35$, $p = 0.005$) and library pinning (OR $= 2.70$, $p = 0.025$) become the strongest correlates.

TABLE IV
LOGISTIC REGRESSION OF MODEL PINNING ($n = 589$)

Predictor	Adj. OR	95% CI	p
CI workflows	1.44	0.84–2.48	0.189
Automated tests	1.96	1.01–3.80	0.046
Docker	1.16	0.71–1.91	0.557
Library-pinned	1.43	0.89–2.30	0.140
$\log_{10}$ stars	1.52	1.13–2.05	0.005
Age (years)	0.98	0.88–1.10	0.777

LR test $p < 0.001$; McFadden pseudo-$R^2 = 0.055$

F. Robustness to Project Popularity

For projects with at least 100, 500, and 1,000 stars (n = 380, 233, 149), the share that never pin falls only to 82.1%, 80.3%, and 75.8%; the asymmetry stays at 77.0–79.6%; and 89.0–93.7% of remote-code sites remain unpinned, while remote-code use rises from 21.7% to 32–33%.

VI. DISCUSSION

What the results mean. Model dependencies are managed by individual choice, not project policy: coverage is low even where projects pin, and pinning concentrates in download functions that expose `revision` prominently. The asymmetry in RQ3 points to a tooling gap rather than a skills gap, since the same teams lock libraries, where lockfile tools make this automatic. In RQ5, tests and popularity, not CI by itself, distinguish pinning projects, and the low R^2_{McF} in (6) shows that current practices explain little of the variation.

Why mature projects still float. We see three plausible reasons; only the third is directly visible in our data, so developer surveys should test them. First, *tooling*: a pin is a 40-character hash copied by hand from the hub, with no lockfile to generate or update it. Second, *perceived stability*: popular checkpoints such as `bert-base-uncased` are widely assumed to be frozen, and updates are assumed to be improvements, so pinning looks like overhead without benefit. Third, *dynamic model selection*: in 47.7% of projects every identifier is indirect, because the model is chosen at run time through arguments or configuration. Frameworks deliberately leave this choice to users, which moves the pinning decision to whoever configures the system, where no tool checks it. Unpinned remote code is a code supply-chain risk as well as a reproducibility risk [14].

Comparison with prior work. Our project-level rate with any revision (15.1%) is close to the 12% that Yasmin et al. found manually [3], which suggests that the practice has not improved since 2023. Our strict (4.4%) and hash-only (1.7%) rates add that a configurable revision protects nothing unless set. Methodologically, RQ1 shows that line-based matching would have yielded a pinning rate near zero.

Recommendation: model lockfiles. We propose committing a `models.lock` file that maps each identifier to a commit hash, loading models through a helper that reads it, and running PinCheck in CI to fail builds containing unpinned loads, unpinned remote code, or default pipelines. A model

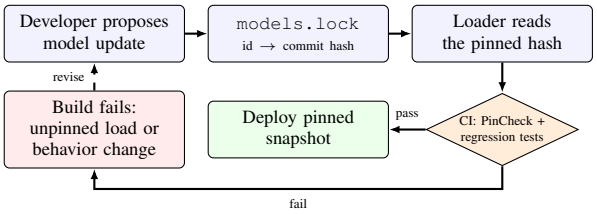


Fig. 5. Proposed model lockfile workflow. Every load reads its commit hash from a version-controlled lockfile, and CI rejects unpinned loads, unpinned remote code, default pipelines, and behavioral regressions before deployment.

update then becomes a reviewable change that can be gated by regression tests [11], [13] (Fig. 5).

Other stakeholders. Libraries could log the resolved commit and warn on unpinned remote code, and hubs could offer immutable numbered releases.

Extending PinCheck. Indirect identifiers usually come from three sources that a future version can resolve: `argparse` defaults and `os.getenv("KEY", default)` fallbacks in the same file; `.env` files; and YAML, JSON, or TOML configuration files such as Hydra configs. A second pass could parse these files, follow the key that the code reads (e.g., `cfg.model.name`), and report whether a matching revision key exists. Inter-procedural propagation would then cover values passed through function parameters.

Other ecosystems. The approach generalizes because the problem does. PyTorch Hub's `torch.hub.load("owner/repo:ref")` resolves a Git branch or tag at download time, so it floats in the same way. TensorFlow Hub handles, by contrast, carry an explicit version number and are pinned by design. Hugging Face clients for JavaScript and Rust accept the same `revision` argument, so the classification rules carry over and only the parser front end must change.

A. Threats to Validity

Construct. Indirect identifiers (78.6% of sites) cannot be resolved statically; some are local paths or are pinned through configuration that PinCheck does not read. Configurable revisions may be unset at run time, hence the strict and hash-only rates. A few download calls fetch datasets rather than models. *Internal.* RQ5 associations are correlational, and the regression explains little variance. *External.* The sample covers public Python projects ($\geq$20 stars, active since 2025) and mixes applications with libraries, for which indirect identifiers are often intentional. *Conclusion.* Primary tests use a Bonferroni correction; strict estimates rest on 26 repositories. *Reliability.* Recall is relative to the union of both tools, and two string-embedded loads were missed during development. A re-collection on 20 October 2026 differed from one on 5 October by at most 0.2 percentage points on project-level pinning rates and 0.8 on practice signals. Only public code was analyzed, no personal data were collected, and PinCheck, the study scripts, and the datasets are publicly available at https://github.com/Aditya-Khetawat/floating-on-main.

VII. Conclusion

We presented PinCheck and showed that syntax-aware detection is necessary to measure model dependency pinning: pattern matching would underestimate it thirteen-fold. In 589 maintained projects, model pinning is rare (84.9% never pin; 1.7% use hashes) and partial (median coverage 16.4%). It is decoupled from library pinning (82.1% asymmetry), and it is absent from most remote-code loads. Future work will measure the behavioral impact of silent updates, resolve configuration-file and environment identifiers, and extend PinCheck to PyTorch Hub and non-Python clients.

Acknowledgment

The authors acknowledge using AI tools for paraphrasing, grammar correction, and improving readability. All research, analysis, and conclusions are the authors' own, who reviewed the revisions and remain responsible for the manuscript's accuracy and content.

References

[1] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (MLOps): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023.

[2] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Proc. NeurIPS*, 2015, pp. 2503–2511.

[3] J. Yasmin, W. Jiang, J. C. Davis, and Y. Tian, "Software dependencies 2.0: An empirical study of reuse and integration of pre-trained models in open-source projects," arXiv:2509.06085, 2025.

[4] A. Ajibode, A. A. Bangash, F. R. Cogo, B. Adams, and A. E. Hassan, "Towards semantic versioning of open pre-trained language model releases on Hugging Face," *Empirical Software Engineering*, vol. 30, 2025.

[5] J. Castaño, S. Martínez-Fernández, X. Franch, and J. Bogner, "Analyzing the evolution and maintenance of ML models on Hugging Face," in *Proc. MSR*, 2024.

[6] "runwayml/stable-diffusion-v1-5 no longer exists," Issue #1305, huggingface/optimum-habana, GitHub, 2024. [Online]. Available: https://github.com/huggingface/optimum-habana/issues/1305

[7] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *Proc. IEEE Big Data*, 2017, pp. 1123–1132.

[8] F. Calefato, F. Lanubile, and L. Quaranta, "A preliminary investigation of MLOps practices in GitHub," in *Proc. ESEM*, 2022, pp. 283–288.

[9] W. Jiang *et al.*, "An empirical study of pre-trained model reuse in the Hugging Face deep learning model registry," in *Proc. ICSE*, 2023, pp. 2463–2475.

[10] W. Jiang *et al.*, "PeaTMOSS: A dataset and initial analysis of pre-trained models in open-source software," in *Proc. MSR*, 2024.

[11] Y. Xie, Y.-A. Lai, Y. Xiong, Y. Zhang, and S. Soatto, "Regression bugs are in your model! Measuring, reducing and analyzing regressions in NLP model updates," in *Proc. ACL*, 2021, pp. 6589–6602.

[12] L. Chen, M. Zaharia, and J. Zou, "How is ChatGPT's behavior changing over time?" *Harvard Data Science Review*, vol. 6, no. 2, 2024.

[13] W. Ma, C. Yang, and C. Kästner, "(Why) is my prompt getting worse? Rethinking regression testing for evolving LLM APIs," in *Proc. CAIN*, 2024.

[14] J. Zhao *et al.*, "Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs," in *Proc. ASE*, 2024.

[15] A. Decan and T. Mens, "What do package dependencies tell us about semantic versioning?" *IEEE Trans. Softw. Eng.*, vol. 47, no. 6, pp. 1226–1240, 2021.

[16] R. G. Kula, D. M. German, A. Ouni, T. Ishio, and K. Inoue, "Do developers update their library dependencies?" *Empirical Software Engineering*, vol. 23, no. 1, pp. 384–417, 2018.

[17] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "A large-scale study about quality and reproducibility of Jupyter notebooks," in *Proc. MSR*, 2019, pp. 507–517.